AgentShield-Crypto: Cryptographic Authorization and Durable Replay Control for Autonomous Multi-Agent Trading Workflows

Saiful Islam Tanvir
AI and Cybersecurity Researcher

***Abstract*—Autonomous workflows built around large language models can transform model-generated recommendations into consequential actions, including financial operations. A central security problem is that natural-language plausibility does not establish the authority, integrity, or freshness of a proposed action. This paper presents AgentShield-Crypto, a lightweight authorization gate that places cryptographic verification and durable replay control between an untrusted proposal source and a simulated execution boundary. The implemented prototype authenticates a risk-decision envelope with HMAC-SHA-256, binds verification to the expected request fields, checks key status and lifecycle policy, and reserves request identifiers in a local SQLite-backed ledger before permitting execution. We evaluate the implementation using a fixed synthetic workload containing 40 valid requests, 30 replay attempts, and 30 signature-tampering attempts per run. Across five repeated runs, the protected pipeline executed all 200 valid requests and none of the 300 unsafe requests; the unprotected baseline executed all 300 unsafe requests. The protected pipeline achieved a mean throughput of 256.303753 requests/s, compared with 55,143.714600 requests/s for the baseline, with mean median latency of 0.470736 ms and 0.013274 ms, respectively. These measurements illustrate the operational cost of the implemented checks in this local test environment, not a general performance guarantee. Additional tests exercise replay handling across processes, concurrent reservation, and key lifecycle transitions. The results support a narrow conclusion: a cryptographic authorization boundary can reject malformed, tampered, replayed, or policy-ineligible requests before a local simulated action is dispatched. They do not establish semantic prompt-injection resistance, production trading safety, distributed exactly-once execution, or generalization to live multi-agent deployments. The paper details the architecture, threat assumptions, experimental procedure, measured results, and limitations needed to reproduce and interpret this prototype responsibly.**

***Index Terms*—LLM agents, applied cryptography, HMAC, replay protection, authorization, SQLite, zero trust, autonomous workflows, security evaluation.**

I. Introduction

Large language models are increasingly used as components in workflows that combine reasoning, tool use, data retrieval, and interaction among multiple agents. Frameworks such as AutoGen illustrate how conversational agents can be composed into broader applications [5]. In a financial workflow, for example, one component may interpret market data, another may formulate a candidate trade, and a downstream service may decide whether to execute it. Such decomposition can improve modularity, but it also creates security boundaries between components with different responsibilities and privileges.

A key distinction is often missed in these systems: a proposal can be syntactically valid and appear plausible without being authorized to cause an external action. Prompt injection research has shown that content retrieved from untrusted sources can influence LLM-integrated applications and their tool use [1]. OWASP likewise identifies prompt injection and excessive agency as important risks for LLM applications [3]. These risks cannot be solved by message formatting alone. Nor does a cryptographic signature establish that the content is factually correct or that the model producing it has reasoned safely.

This paper investigates a narrower, complementary control. Instead of attempting to determine whether a model's natural-language reasoning is correct, AgentShield-Crypto places a verifiable authorization gate at the boundary where a proposed action becomes eligible for dispatch. The gate validates a message-authentication code, checks request binding and key policy, and records request consumption durably before allowing the local execution stub to run. A request that fails any required check is rejected before that stub is invoked.

The contribution is an implemented prototype and an empirical evaluation of this gate under a controlled synthetic workload. The work is intentionally not presented as a complete multi-agent platform or a production trading system. In the current implementation, HMAC-SHA-256 authenticates a bounded risk-decision envelope; the evaluation uses local Python components and SQLite; and the final action is simulated. Prompt-state hashing, vector clocks, Byzantine consensus, zero-knowledge proofs, and cross-host coordination are not treated as implemented capabilities.

The paper makes four contributions:

- It specifies a practical authorization boundary for a proposed high-impact action, separating message authenticity and policy eligibility from the semantic correctness of an LLM output.
- It describes the prototype's HMAC-based envelope verification, request binding, key lifecycle checks, and durable replay reservation.
- It reports repeated results for a fixed workload and complementary lifecycle/concurrency tests, including throughput and latency trade-offs against an unprotected baseline.
- It provides a threat model and limitations that distinguish what the implementation demonstrates from what would require further testing in a distributed or live trading environment.

II. Background and Related Work

A. Prompt injection and agent authority

Prompt injection arises when an LLM processes adversarial instructions embedded in user input or external content. Indirect prompt injection is particularly relevant to tool-enabled systems because the attacker may influence data that the model later retrieves rather than interact with the model directly [1]. The resulting risk depends on the application architecture: an injected instruction becomes consequential when the system gives model output access to privileged tools or treats a model-generated proposal as sufficient authorization.

AgentShield-Crypto does not attempt to identify every malicious instruction or classify semantic intent. It instead limits the authority of a proposal at a downstream boundary. This distinction matters. An authenticated message can still contain an unsafe decision if an authorized component signs it, if a signing key is compromised, or if the policy itself is defective. Cryptographic checks address authenticity and integrity under stated assumptions; they are not a substitute for model evaluation, data validation, or independent risk controls.

B. Zero trust and authorization boundaries

NIST's zero-trust architecture guidance rejects implicit trust based only on network location or ownership and emphasizes explicit authentication and authorization before access to protected resources [2]. The prototype applies this general principle to a proposed action: the request is not trusted merely because it came from a known local workflow or contains a plausible payload. It must pass the gate's checks before dispatch. This is an application-level design choice, not a claim that the prototype implements a complete enterprise zero-trust architecture.

C. Message authentication and durable replay control

HMAC provides message authentication using a shared secret and a cryptographic hash function [4]. It can detect unauthorized modification of authenticated fields, assuming the key remains secret and the implementation verifies the expected message representation correctly. HMAC does not provide confidentiality, and because the key is shared, it does not provide the same public verifiability as a digital signature.

Replay prevention requires more than checking whether a message is well formed. A previously valid authenticated request may be resent. Timestamp windows can reduce the useful lifetime of a message, but a request can be replayed within the window unless the receiver also tracks consumption or uses an equivalent freshness mechanism. AgentShield-Crypto therefore combines envelope validation with a durable reservation of request identifiers. The SQLite ledger is a local prototype mechanism; its use does not by itself establish correctness under all distributed database, failover, or multi-host conditions.

D. Multi-agent orchestration

Multi-agent frameworks provide mechanisms for composing agents and tools into larger workflows [5]. Such orchestration does not automatically provide end-to-end authorization for downstream side effects. AgentShield-Crypto is designed as a narrow gate that could sit at an application boundary; this paper does not claim compatibility with every orchestration framework, nor does it evaluate model-level behaviour across different LLM providers.

III. System and Threat Model

A. Protected operation

The evaluated workflow receives a proposed action represented by a structured risk-decision envelope. The gate checks whether the envelope is authentic and eligible under local policy, reserves the request identifier, and only then invokes a simulated execution function. The execution function is a test stub rather than a broker, exchange, or financial institution API.

Let a request be represented by

$$x = (r, a, q, p, k, \tau, \sigma), \tag{1}$$

where r is a request identifier, a denotes the relevant agent or producer identity, q is the structured decision payload, p is the policy-relevant binding data, k identifies the verification key, τ is freshness metadata, and σ is the HMAC tag. The precise fields included in the authenticated representation must be the same at signing and verification. The prototype validates the implemented envelope fields; it should not be interpreted as authenticating hidden model state or an entire conversation history.

The authorization predicate is expressed at a high level as

$$\mathsf{Allow}(x) = \mathsf{Auth}(x) \wedge \mathsf{Bound}(x) \wedge \mathsf{KeyOK}(k) \wedge \mathsf{Fresh}(x) \wedge \mathsf{PolicyOK}(x) \wedge \mathsf{Re} \tag{2}$$

Here, $\mathsf{Reserve}(r)$ means the request identifier was successfully reserved according to the local durable ledger's uniqueness and transaction rules. Equation (2) is a conceptual description of the implemented decision sequence, not a formal proof of atomicity across the gate and an external execution service.

B. Adversary capabilities

The threat model includes an attacker who can submit malformed requests, alter authenticated fields, resend a previously observed request, or race multiple submissions for the same request identifier. The attacker may also attempt to use a key that is expired, revoked, or otherwise ineligible under the prototype's lifecycle policy. These capabilities align with the replay, tampering, and lifecycle scenarios exercised in the tests.

The following are outside the demonstrated threat model: compromise of an authorized signing key before signing; malicious or compromised code inside the trusted gate; operating-system or host compromise; SQLite corruption; arbitrary modification of the trusted policy store; manipulation of the Python runtime; and attacks against a live external broker. The experiments also do not measure whether an LLM can be induced to generate semantically harmful but correctly authenticated proposals.

C. Security objectives and non-objectives

The intended objectives are (i) reject a request whose authentication tag or bound fields are invalid; (ii) reject duplicate consumption of a request identifier under the tested ledger conditions; (iii) apply key lifecycle policy before dispatch; and (iv) ensure rejected requests do not invoke the simulated execution stub. The implementation is not intended to guarantee market correctness, detect all prompt injections, prevent a compromised authorized agent from signing an unsafe action, or provide distributed exactly-once execution.

IV. AgentShield-Crypto Design

A. Processing stages

The prototype places a gate between request submission and a local execution stub. Fig. 1 summarizes the logical stages. The diagram describes the experimental implementation at a high level and should not be read as a deployment topology tested across multiple hosts.

The sequence is deliberately ordered so that an authentication or policy failure does not reach the execution stub. The durable reservation is a separate control from the HMAC check: an authentic request can still be a replay. The gate records relevant outcomes for evaluation, but the paper does not claim that its local audit log is tamper-proof against a host-level attacker.

B. HMAC envelope verification

HMAC-SHA-256 is used as a message authentication code over the prototype's canonical envelope representation. At verification time, the receiver recomputes the tag using the selected key and compares it with the received tag using the implementation's verification routine. The verifier also checks that the fields in the incoming request match the fields expected by the application context. This prevents an attacker from changing a bound field without invalidating the authentication tag, assuming the field is actually included in the authenticated representation.

Canonicalization is security-relevant. If signing and verification serialize the same logical request differently, valid requests may be rejected; if fields are omitted from the authenticated representation, they may be changed without detection. For deployment, the representation should use an explicitly versioned schema, deterministic serialization, unambiguous field boundaries, and test vectors covering encoding edge cases. These are engineering requirements, not claims that every possible serialization edge case has been formally verified here.

C. Authorization predicate and state transitions

Let an envelope be $e = (d, k, r, \sigma)$, where d is the bound risk decision, k is the key identifier, r is the unique request identifier, and σ is the authentication tag. Let $\mathsf{Eligible}(k, t, P)$ mean that key k is active at time t and permitted by policy P. Let $\mathsf{Bind}(d, c)$ indicate that the decision is bound to the expected proposal context c. The gate's acceptance predicate is

$$\begin{aligned}\mathsf{Accept}(e, c, t) = \mathsf{Schema}(e) &\wedge \mathsf{Eligible}(k, t, P) \\ &\wedge [\sigma = \mathsf{HMAC}_{K_k}(\mathsf{Canon}(d, c, r))] \\ &\wedge \mathsf{Bind}(d, c) \wedge \mathsf{Policy}(d, P) \\ &\wedge \mathsf{Reserve}(L, r).\end{aligned} \tag{3}$$

Here $\mathsf{Reserve}(L, r)$ is true only when the ledger commits a previously unseen request identifier. The conjunction is important: no single term substitutes for the others. HMAC authenticates bytes under a secret; it does not establish the truth of a decision, key eligibility, request freshness, or safe downstream semantics. The implementation tests exercise these checks in a local prototype; Eq. (3) is a specification of the intended decision boundary, not a claim of machine-checked refinement between specification and every code path.

The logical state progression is $\mathsf{Received} \rightarrow \mathsf{Verified} \rightarrow \mathsf{PolicyApproved} \rightarrow \mathsf{Reserved} \rightarrow \mathsf{Dispatched}$. A failure before reservation terminates in $\mathsf{Rejected}$ without dispatch. A failure after reservation can leave a consumed identifier with no confirmed side effect; this is intentionally not relabelled as a successful execution. The prototype's tests demonstrate rejection and reservation behaviours, but the downstream side effect is simulated and is not transactionally coupled to SQLite.

D. Algorithmic properties and cost model

For a payload of n bytes, HMAC-SHA-256 verification requires work linear in the authenticated payload size, $O(n)$, while parsing and canonicalization are also at least linear in the serialized input. With a B-tree-backed unique index over N consumed identifiers, an individual lookup/insert is typically $O(\log N)$; actual latency depends on transaction mode, journaling, filesystem, contention, and audit work. Thus a useful first-order model is $T(n, N) = T_{\mathrm{parse}}(n) + T_{\mathrm{mac}}(n) + T_{\mathrm{policy}} + T_{\mathrm{reserve}}(N) + T_{\mathrm{dispatch}}$. The benchmark reports end-to-end harness timing and does not isolate these terms, so the model explains likely cost sources but does not assign measured costs to individual components.

Conditional safety argument. If (i) the secret key remains unknown to the adversary, (ii) the verifier authenticates the same canonical fields that are used for dispatch, (iii) key policy and request binding are checked before reservation, and (iv) the unique reservation is atomic in the tested database, then a modification of authenticated fields should fail verification except with negligible MAC-forgery probability, and a second reservation of the same identifier should fail under that database's uniqueness semantics. This is a conditional argument about the specified controls. It does not cover a compromised signing key, a malicious but authorized decision, host compromise, database rollback, multiple unsynchronised ledgers, or a side effect that occurs outside the reservation transaction.

E. Key lifecycle

The implementation includes lifecycle-aware checks for key eligibility, including rotation, revocation, and expiry scenarios

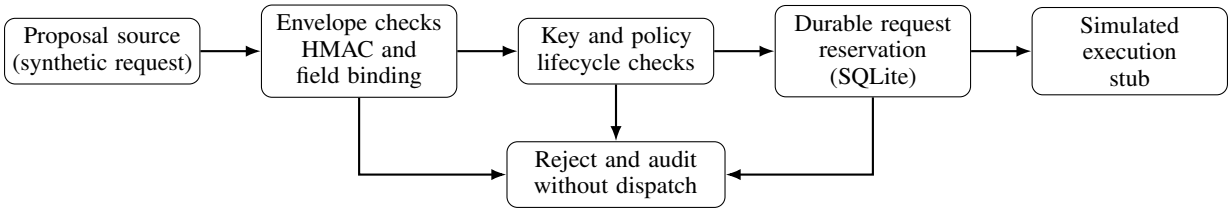


Fig. 1. Logical authorization flow in the local prototype. A request is dispatched to the simulated execution stub only after verification, policy checks, and successful request reservation.

exercised by the test suite. A key identifier is not itself a secret and must not be treated as proof of authorization. The verifier must resolve the identifier through trusted policy state and reject requests signed with keys that are not eligible for the relevant operation. Key lifecycle management in this prototype is local; secure provisioning, hardware-backed storage, remote rotation coordination, and compromise recovery in a distributed deployment remain future work.

F. Durable replay reservation

A valid message should not be accepted repeatedly simply because its HMAC remains valid. The prototype therefore reserves request identifiers in a SQLite-backed ledger. Under the tested local conditions, uniqueness constraints and transaction handling are used to prevent multiple submissions for the same identifier from all proceeding as new requests. Tests include sequential replays across independent processes and simultaneous local reservation attempts.

The reservation protects the gate's decision to accept a request; it does not solve the classic distributed side-effect problem by itself. A process could reserve a request and then crash before dispatch, or an external service could execute a request while the caller fails before recording completion. Conversely, retrying an external operation may duplicate a side effect if the external service does not support idempotency. A production system would need an explicit state machine, idempotency support at the execution service, recovery procedures, and carefully designed transaction boundaries.

G. Reservation semantics and recovery boundary

The reservation operation is the linearization point for the local single-use decision: concurrent contenders for the same identifier should not both observe a successful first reservation if the unique insert is correctly committed. This property is narrower than exactly-once execution. A safe integration needs an explicit lifecycle record such as NEW, RESERVED, DISPATCHING, CONFIRMED, and FAILED_AMBIGUOUS. The present tests support the reservation and duplicate-rejection portions of this state model; they do not establish a fully implemented recovery protocol for all states.

Algorithm 1 intentionally distinguishes tested local reservation behaviour from the deployment design needed around an external executor. The idempotency and reconciliation lines are requirements for a future integration, not features claimed as validated by the present prototype.

Algorithm 1 Single-use reservation and retry interpretation

Require: Durable ledger L, request identifier r
Ensure: Reservation result and safe retry classification
Begin database transaction
if CONTAINS(L, r) **then**
  Roll back transaction
  return (duplicate, do-not-dispatch)
end if
Insert r with state RESERVED under a unique constraint
if insert or commit fails **then**
  Roll back if possible
  return (reservation-failed, do-not-dispatch)
end if
Commit transaction
Dispatch once using stable idempotency key r ▷ deployment requirement
if executor confirms the effect **then**
  Record CONFIRMED
  return (confirmed, no-retry-needed)
else
  Record or reconcile FAILED_AMBIGUOUS
  return (ambiguous, reconcile-before-retry)
end if

V. Implementation and Experimental Method

A. Prototype scope

The implementation is a local Python prototype with SQLite-backed durable state and a simulated execution boundary. The benchmark compares a protected path with an unprotected baseline using the same fixed synthetic workload. The baseline is intended to expose the difference in request handling when the gate's rejection checks are absent; it is not a comparison against another production security product or a neural guardrail.

B. Workload

Each run contains 100 requests: 40 valid requests, 30 replay attempts, and 30 tampered-signature requests. The same workload composition is repeated five times. Thus, the evaluation covers 500 request observations in the protected path and 500 in the baseline, with 200 valid requests and 300 unsafe requests per path across the five runs. The use of a fixed synthetic workload supports controlled comparison, but limits the diversity of attacks and the statistical population to which results can be generalized.

The principal security outcome is whether a request is executed by the simulated downstream function. Valid execution rate is the number of valid requests executed divided by the number of valid requests presented. Unsafe execution rate is the number of replay or tampering requests executed divided by the number of unsafe requests presented. Throughput is measured in requests per second under the harness's timing method. Latency is summarized by the reported median, 95th percentile, and 99th percentile. The reported aggregate figures are the means across five repeated runs; they should not be interpreted as measurements from five independent production deployments.

C. Additional lifecycle and concurrency checks

Beyond the repeated fixed-workload benchmark, separate tests exercised replay across independent local processes, simultaneous local SQLite reservation attempts, key rotation, revocation and expiry, and failure/retry paths. The combined lifecycle test suite reported 12 of 12 checks passing. These checks are useful functional evidence, but they are not a substitute for formal verification, stress testing under arbitrary process scheduling, or fault injection across a distributed infrastructure. They do not establish a universal bound on race conditions or prove end-to-end atomicity with an external execution system.

D. Measurement protocol and reporting requirements

The final report aggregates five repeated runs and reports the arithmetic mean of the per-run throughput and latency summaries. Because the workload contains only 100 requests per run, the resulting latency percentiles should be understood as descriptive summaries of a small harness run. In particular, a $p99$ estimate from 100 observations is close to an extreme observation and can be sensitive to scheduling, storage activity, and measurement overhead. A more complete performance study would increase the request count substantially, randomize the order of cases, separate warm-up from measurement, and repeat the test under controlled concurrency levels.

A reproducibility package should identify the Python version, dependency versions, operating system, processor, storage medium, SQLite settings, and exact command used to run the harness. It should include the request-generation logic, expected counts, per-run output, and a clear description of which time interval contributes to throughput. When publishing the package, secrets and production credentials must be excluded, and all test keys should be clearly labelled as non-production. The evidence archive produced by the prototype is a starting point for these practices; the manuscript does not assume that the archive alone provides independent reproduction by another research group.

For security evaluation, a second dimension of reproducibility is attack diversity. The current workload intentionally isolates two integrity-related failure classes: replay and signature tampering. Future cases should separately cover unknown key identifiers, malformed encodings, field substitution, stale and expired requests, policy-boundary violations, duplicate identifiers under contention, and recovery after process termination. Those tests should be reported separately rather than merged into a single headline mitigation rate, because each case exercises a different control and has a different failure mode. The current lifecycle tests cover a subset of these behaviours, but the paper does not claim comprehensive attack-family coverage.

Algorithm 2 Fail-closed authorization and dispatch

Require: Envelope x, key registry K, policy P, replay ledger L
Ensure: Reject, or one simulated dispatch attempt

$e \leftarrow \text{PARSEANDVALIDATESCHEMA}(x)$
if $e = \perp$ **then**
  return REJECT(malformed)
end if
$k \leftarrow K[e.\text{key_id}]$
if $k = \perp$ or $\neg\text{KEYELIGIBLE}(k, P)$ **then**
  return REJECT(key-policy)
end if
$m \leftarrow \text{CANONICALPAYLOAD}(e.\text{decision}, e.\text{binding})$
$t \leftarrow \text{HMAC_SHA256}(k.\text{secret}, m)$
if $\neg\text{CONSTANTTIMEEQUAL}(t, e.\text{tag})$ **then**
  return REJECT(authentication)
end if
if $\neg\text{BINDINGVALID}(e, P)$ or $\neg\text{POLICYALLOWS}(e, P)$ **then**
  return REJECT(binding-policy)
end if
if $\neg\text{RESERVEONCE}(L, e.\text{request_id})$ **then**
  return REJECT(replay)
end if
$r \leftarrow \text{SIMULATEDEXECUTE}(e.\text{decision})$
AUDITOUTCOME(e.request_id,r)
return r

E. Reproducibility and interpretation

The experiment recorded a final evidence report, reproducibility manifest, CSV benchmark table, figures, and a packaged evidence archive. The numerical results below are taken from the final integrated evaluation output. The report's final status was `PASS_WITH_LIMITATIONS`. No external exchange, broker, real financial account, or live LLM inference service was used in the final benchmark described here. Consequently, the results are best interpreted as a controlled prototype evaluation of the authorization gate rather than an evaluation of model quality or financial performance.

F. Decision procedure and failure handling

Algorithm 2 states the gate's intended control flow in implementation-neutral form. It makes explicit that authentication is necessary but not sufficient: key status, policy checks, and request reservation are separate decisions. A rejection at any stage terminates the request before the simulated execution function is called. The pseudocode is a specification of the evaluated control boundary, not a formal proof that all runtime paths implement it without defect.

The ordering of steps is a security and operational trade-off. Parsing and key lookup are required before meaningful verification can occur. Reservation should happen only after the request is authenticated and policy-eligible, otherwise unauthenticated traffic could consume valid identifiers. However, reservation before dispatch creates a recovery problem if dispatch fails after the identifier is consumed. The current benchmark measures rejection and local execution outcomes; it does not claim to resolve this failure window. A production state machine must distinguish a request that has been accepted from one whose side effect has been confirmed.

G. Failure semantics and auditability

A security gate should make rejection observable rather than silently converting failures into successful-looking outputs. In the evaluated run, the evidence report recorded zero audit failures and zero request-audit failures. This is a measured property of the finite test run, not an availability guarantee. A production implementation should define behaviour for a full disk, a locked database, corrupted state, an unavailable key service, or an audit sink that cannot accept a record. For high-impact operations, a fail-closed default is often appropriate, but it may also reduce availability; that trade-off should be explicitly set by the application owner rather than left to incidental exception handling.

Audit events should avoid storing secret key material and should include enough non-sensitive context to support investigation: a stable request identifier, decision outcome, reason code, key identifier, policy version, and timestamps from a documented clock source. If payloads may contain confidential market or user data, event schemas should prefer hashes or carefully selected metadata over unrestricted payload logging. The prototype's local audit checks do not establish tamper-evident logging, confidentiality of logs, or protection against a privileged host attacker.

H. Threat-to-control mapping

Table I maps the evaluated attack classes to the relevant control and identifies what remains outside the evidence. This mapping prevents the observed rejection rate from being interpreted as a general prompt-injection defence rate.

TABLE I
THREAT-TO-CONTROL MAPPING AND EVIDENCE BOUNDARY

Threat	Relevant control	Evidence boundary
Modified authenticated field	HMAC verification and request binding	Tampering requests rejected in fixed workload; no exhaustive encoding analysis
Previously consumed request	Durable unique reservation	Local sequential and concurrent tests; no distributed ledger proof
Revoked or expired key	Lifecycle eligibility check	Enumerated local lifecycle tests passed; remote propagation untested
Duplicate concurrent submission	Transactional reservation	Local SQLite/process tests; external side effect not atomic
Compromised authorized key	None sufficient in current gate	Out of scope; valid MAC may authenticate unsafe content
Semantic prompt injection	Independent semantic and business controls needed	Not tested by this benchmark
Host or policy-store compromise	Trusted computing base assumption	Out of scope

VI. RESULTS

A. Request acceptance and execution

Table II reports the aggregate execution outcomes over five repeated runs. The protected pipeline executed all 200 valid requests and rejected all 300 unsafe requests before the simulated execution boundary. The unprotected baseline executed all 300 unsafe requests. For this fixed workload, the observed unsafe execution rate was therefore 0% for the protected path and 100% for the baseline.

The observed rates are exact counts for the tested inputs. They are not estimates of performance across an unknown population of attacks, and the fixed-workload bootstrap interval of 0–0 for the protected unsafe-acceptance rate and 1–1 for the baseline merely reflects resampling from repeated identical workload composition. It must not be presented as a population-level confidence interval or evidence that every attack class will be blocked.

TABLE II
AGGREGATE OUTCOMES OVER FIVE REPEATED RUNS

Measure	Protected	Baseline
Total requests	500	500
Valid requests presented	200	200
Valid requests executed	200	200
Unsafe requests presented	300	300
Unsafe requests executed	0	300
Observed unsafe execution rate	0%	100%
Execution/audit exceptions reported	0	0

B. Run-level consistency

The final integrated report gives the same request composition in each of the five repetitions: 40 valid requests, 30 replay attempts, and 30 tampered-signature attempts. The protected result was 40 valid executions and zero unsafe executions in each run; the baseline result was 40 valid executions and 60 unsafe executions in each run. Table III shows these counts. The repeated outcome is useful as a deterministic regression check for the specific workload, although repetition of the same fixed cases is not equivalent to sampling independent attack scenarios.

The table separates the number of unsafe requests presented from the number executed. This matters because a rejection-only metric can conceal an over-restrictive system that rejects legitimate requests as well. In this workload, the valid-request execution rate was 100% for both paths, while the unsafe-request execution rate differed. The conclusion is restricted to the supplied valid and unsafe test cases; no claim is made about false-positive behaviour on a broad distribution of legitimate financial actions.

C. Throughput and latency

Table IV summarizes the reported mean performance metrics. The protected path's mean throughput was 256.304 requests/s, while the baseline's mean was 55,143.714600 requests/s. The

TABLE III
PER-RUN EXECUTION COUNTS FOR THE FIXED WORKLOAD

Run	Valid in	Valid exec.	Unsafe in	Unsafe exec.
1	40	40	60	0 / 60
2	40	40	60	0 / 60
3	40	40	60	0 / 60
4	40	40	60	0 / 60
5	40	40	60	0 / 60

TABLE IV
MEAN PERFORMANCE ACROSS FIVE REPEATED RUNS

Metric	Protected	Baseline
Throughput (requests/s)	256.304	55,143.715
Median latency (ms)	0.471	0.013
$p95$ latency (ms)	11.297	0.019
$p99$ latency (ms)	14.787	0.043

protected path's mean median latency was 0.471 ms, compared with 0.013 ms for the baseline. The protected path also reported mean $p95$ latency of 11.297 ms and mean $p99$ latency of 14.787 ms; the corresponding baseline values were 0.019 ms and 0.043 ms.

These results show a substantial measured performance cost for the protected path under the current harness. The throughput ratio is approximately 194.8:1 in favour of the baseline. This is not an apples-to-apples comparison between security products: the baseline omits the security checks whose cost is being measured, and the test environment and harness may contribute to the absolute values. The latency tail is also much larger in the protected path than its median, suggesting that repeated-run averages alone are insufficient to characterize operational responsiveness. A production assessment should report raw per-request samples, warm-up policy, machine details, database configuration, concurrency level, and confidence intervals across independent runs.

D. Effect size within the fixed workload

For the tested unsafe subset, the empirical execution-rate difference is $1.00 - 0.00 = 1.00$ (100 percentage points) in favour of the protected gate. This is a deterministic contrast on 300 labelled unsafe requests, not a population effect estimate. For valid requests, both paths execute $200/200 = 1.00$; hence the measured difference in valid execution rate is zero percentage points for this dataset. These two quantities should be reported together: a system that rejects every request could achieve zero unsafe executions but would fail the legitimate-request objective. The current test set shows no valid-request loss in the supplied cases, but its small, constructed composition cannot establish a low false-rejection rate in deployment.

The mean throughput ratio is $55143.714600/256.303753 \approx 215.1$ in favour of the baseline, based on the exact final integrated report. This ratio describes the measured harness and should not be called a security overhead factor in isolation because the baseline omits the checks that define the protected treatment. The reported mean $p99$ latency is also sensitive to a small sample: with 100 requests per run, tail estimates are coarse and can be influenced by a single scheduling or storage pause. The correct next step is component-level profiling and a larger randomized benchmark, not treating these numbers as universal service-level predictions.

E. Functional lifecycle and concurrency results

The supplementary test suite passed 12 of 12 lifecycle-oriented checks. These tests covered combinations of durable replay handling, key rotation, revocation, expiry, and local concurrent reservation. The local concurrent reservation experiment also exercised eight subprocesses attempting reservation under the same scenario. These outcomes support the claim that the tested prototype handled the enumerated local cases. They do not prove that SQLite transactions provide the same behaviour under every filesystem, network storage, failover, or distributed deployment configuration.

F. Summary of evidence

The final evaluation reported zero audit failures and zero request-audit failures for the measured workload. These are useful operational checks because a security gate should not silently lose the evidence needed to explain its decisions. Nevertheless, a zero-failure observation over a finite local run does not prove that audit logging cannot fail. The reported final status, `PASS_WITH_LIMITATIONS`, is therefore an appropriate summary: the enumerated tests passed, while important deployment and threat-model limitations remain.

G. Metric definitions and statistical caution

For clarity, let V be the number of valid requests presented, V_e the number of valid requests executed, U the number of unsafe requests presented, and U_e the number of unsafe requests executed. We report valid execution rate as V_e/V and unsafe execution rate as U_e/U. These are execution outcomes rather than a model classifier's precision or recall: the gate is not scoring natural-language intent, and the test labels are assigned by construction. The term "unsafe" in the tables refers specifically to the replay and signature-tampering cases in the workload.

Throughput is reported as the benchmark harness's number of processed requests divided by elapsed time. Latency percentiles describe the request timings collected by the harness; the summary uses the reported mean of each run's median, $p95$, and $p99$ rather than pooling every individual request into one distribution. These summaries answer a different question from the median of all requests pooled together. For a follow-up study, we recommend publishing raw observations and reporting both per-run summaries and pooled quantiles, with the measurement interval and treatment of warm-up clearly specified.

The five repetitions are repeated executions of the same workload structure and do not provide five independent samples from the space of attacks. The reported bootstrap intervals for unsafe acceptance were calculated by resampling the fixed repeated-workload observations. Since all observed protected outcomes were zero unsafe executions and all baseline

outcomes were 60 unsafe executions per run, the resulting intervals remain at their observed boundaries. Such intervals describe variability under that resampling scheme; they do not bound the probability of an unknown future attack succeeding. A stronger estimate of generalization would require diverse independently generated cases, randomized workload order, multiple seeds, and explicit coverage of attack subtypes.

H. Ablation evidence and causal limits

The notebook's isolated ablation harness reports that authentication and replay controls each matter to the tested rejection behaviour. The combined functional suite reports 16 of 16 checks passing, and the later durable lifecycle suite reports 12 of 12 checks passing. These counts should not be added together as if they were independent trials: they are different test suites with overlapping purposes, and their cases are designed regression checks rather than random samples. Nor should the ablation result be interpreted as a precise estimate of each control's marginal contribution in production. It supports a design-level observation that authentication and stateful single-use enforcement address different failure modes: HMAC rejects altered authenticated content, while the ledger rejects repeated use of an otherwise authentic request.

VII. Security Analysis

A. What the HMAC check establishes

Assuming the secret key is protected, the authenticated representation is unambiguous, and the verifier correctly recomputes the tag, a party without the key should not be able to modify authenticated fields and produce a valid HMAC except with negligible success probability under the security assumptions of the construction [4]. This is a conditional statement about message authentication. It does not establish that the request is semantically safe, that the signing agent is uncompromised, or that the key has not been disclosed.

B. Replay and request uniqueness

A replayed message can retain a valid authentication tag. The replay control therefore depends on the stateful reservation mechanism rather than HMAC alone. The observed tests indicate that duplicate request identifiers were rejected under the tested local conditions. If request identifiers can be regenerated or reused, if the ledger can be rolled back, or if multiple hosts use independent ledgers, the same property may not hold. A distributed deployment needs shared or partition-aware replay state and a carefully specified recovery model.

C. Key lifecycle enforcement

The key lifecycle checks reduce the chance that a request signed by an ineligible key reaches dispatch. Their effectiveness depends on the integrity and freshness of the policy state used by the verifier. If an attacker can modify the trusted key registry or compromise an authorized key, the checks may be bypassed. A production implementation should isolate key material, restrict administrative privileges, record lifecycle transitions, and define the behaviour of requests that race with revocation.

D. Semantic compromise remains possible

An authorized component may sign a proposal that is wrong, manipulated by prompt injection, based on poisoned market data, or inconsistent with business rules not represented in the gate. The cryptographic layer can establish that a message matches an authenticated envelope; it cannot infer whether a trade is economically sensible. This limitation is central rather than incidental. The control is intended to complement semantic safeguards, independent risk checks, constrained tool permissions, human approval for high-impact actions, and validation of data sources.

E. Reservation is not external exactly-once execution

A successful reservation followed by a process crash can leave a request marked as consumed even though the external action did not occur. If the system retries without a reconciliation protocol, it may either lose an action or duplicate it. Since the prototype's execution boundary is simulated, the experiments do not address this distributed transaction problem. A production design should use a durable state machine such as `PENDING`, `RESERVED`, `DISPATCHED`, `CONFIRMED`, and `FAILED`, together with downstream idempotency keys and reconciliation of ambiguous outcomes.

VIII. Discussion

The principal result is not that cryptography solves prompt injection. It is that an explicit authorization boundary can enforce a limited set of integrity, freshness, and policy conditions before a request reaches a simulated side effect. This is useful because security responsibilities become clearer: model reasoning generates a proposal, application policy decides whether the proposal is eligible, and a separate gate verifies the request representation and consumption state.

The performance results also highlight an engineering trade-off. In the current prototype, the protected path is materially slower than the unprotected baseline. The measured overhead should not be hidden behind a headline claim of sub-millisecond protection: while the mean median is approximately 0.471 ms, the reported mean $p95$ and $p99$ are approximately 11.297 ms and 14.787 ms. Further profiling is required to identify the sources of tail latency, distinguish cryptographic computation from database contention and harness overhead, and evaluate optimizations without weakening security checks.

The benchmark is intentionally small and fixed. Its advantage is interpretability: the expected number of valid, replayed, and tampered requests is known, so the observed counts can be checked directly. Its limitation is external validity. The workload does not represent the full diversity of real-world prompt injection, model compromise, key theft, network faults, or adversarial scheduling. A broader evaluation should include multiple workload seeds, independently generated attack families, variable concurrency, storage and crash faults, and a real downstream service with documented idempotency semantics.

The title's reference to autonomous trading describes the intended application context, not a claim that live trades were performed. The experimental system did not connect to an

exchange or broker, did not use real money, and did not evaluate profitability, slippage, market risk, or financial compliance. The current evidence is therefore most relevant to application-layer authorization controls for high-impact agent workflows.

IX. Limitations and Future Work

Several limitations should guide interpretation. First, the benchmark uses synthetic requests and a local prototype. Second, the evaluation does not use live LLMs, so it does not quantify how frequently a model produces unsafe proposals or whether model behaviour changes under attack. Third, the local SQLite ledger does not establish multi-host consistency or distributed exactly-once side effects. Fourth, the results do not constitute a formal verification of the complete implementation. Fifth, absolute latency and throughput depend on the execution environment and benchmark harness, details of which should be preserved in the reproducibility package and reported alongside any future rerun.

The next evaluation should publish the source code and environment manifest, include raw per-run results and a fully specified timing protocol, test larger workloads and independent seeds, and compare configurations with individual checks disabled only where the resulting security meaning is clearly described. A separate integration study should connect the gate to a sandboxed downstream service that supports idempotency keys, then inject crashes at each state transition to measure recovery correctness. A multi-host study should evaluate shared replay state, clock assumptions, key distribution, revocation propagation, and network partitions. Finally, semantic safeguards should be evaluated separately from cryptographic verification so that the two layers are not conflated.

A. Deployment questions that remain open

Moving from the prototype to a service that protects real actions would require resolving several questions before any live use. First, the identity and key-management model must be defined: who provisions each key, which principals can authorize a key rotation, how key compromise is reported, and how revocation is enforced when components are disconnected. Second, the authenticated schema must be versioned and documented. A policy version or risk-limit identifier should be included where it affects the authorization decision, so that a request cannot be reinterpreted under an unintended policy. Third, request identifiers must be unique within a well-defined namespace and retained for a period consistent with retry and replay risks.

Fourth, the execution service should participate in idempotency. A gate can record that a request was reserved, but only the downstream system can reliably identify whether a side effect was applied if a network timeout occurs after dispatch. One practical design is to send a stable idempotency key to the executor and reconcile the executor's recorded outcome before retrying. This still requires careful handling of ambiguous failures and cannot be inferred from the current simulated executor. Fifth, observability must be treated as part of the security design: metrics should expose rejection reasons and latency distributions without leaking secrets, and logs should be protected against unauthorized alteration. Finally, the deployment should undergo independent security review and fault-injection testing before being used for real financial operations.

These requirements are not implementation claims. They identify the engineering evidence needed to bridge the gap between a local proof of concept and a dependable production control. Reporting them explicitly helps prevent a prototype result from being interpreted as operational certification.

B. Interpreting the security–performance trade-off

The performance comparison should be interpreted as a cost measurement for the checks enabled in this prototype, not as evidence that the baseline is a suitable alternative for sensitive operations. The unprotected path is faster in this harness precisely because it does not perform the same verification and reservation work. In a high-impact application, the relevant engineering question is not whether security can be removed to maximize throughput; it is which checks are required, where their cost arises, and whether their implementation can be improved without weakening the policy boundary.

Several optimizations could be evaluated in a controlled follow-up. The verifier could reuse parsed, canonical request representations within a single request lifecycle; database indexes and transaction duration could be profiled; and contention could be varied independently of cryptographic verification. Batching may improve throughput in some systems but could increase the time a request waits before authorization, so both throughput and tail latency must be measured. Any optimization that caches key eligibility or policy state introduces a freshness question: the evaluation should explicitly test whether revocation and expiry take effect within the required bound. Performance engineering must preserve the semantics of rejection, replay reservation, and lifecycle enforcement.

The current mean median latency is lower than the reported mean tail latencies by more than an order of magnitude. This suggests that averages alone would provide an incomplete operational picture. A service-level objective should be defined using tail behaviour under realistic concurrency and load, and overload handling should be tested separately. The existing numbers are sufficient to identify the need for further profiling, but not to choose a deployment capacity or claim suitability for latency-sensitive trading. No claim about profitable execution, market responsiveness, or exchange-level performance follows from a local synthetic benchmark.

X. Conclusion

This paper presented AgentShield-Crypto, a local prototype that combines HMAC-based request authentication, policy and key lifecycle checks, and durable request reservation before a simulated action is executed. In a fixed workload repeated five times, the protected pipeline executed 200 of 200 valid requests and zero of 300 replay or tampering requests, while the unprotected baseline executed all 300 unsafe requests. The protected path was substantially slower in the measured harness, with mean throughput of 256.303753 requests/s and mean

median latency of 0.471 ms. Supplementary local lifecycle and concurrency tests passed the 12 enumerated checks.

The evidence supports a bounded conclusion: under the tested local conditions, the gate rejected the enumerated malformed, replayed, tampered, or policy-ineligible requests before simulated dispatch. It does not establish protection against all prompt injections, compromise of authorized keys, production trading safety, or distributed exactly-once execution. Those questions require broader experiments and a real downstream integration. By making the trust boundary explicit and reporting both successful checks and unresolved limitations, AgentShield-Crypto provides a reproducible starting point for evaluating cryptographic authorization controls in autonomous workflows.

Appendix A
Reproducibility and Evaluation Checklist

This appendix records the minimum information needed to interpret a rerun of the reported experiment. It is included because security results are difficult to assess when a headline rate is separated from the exact request composition, trust assumptions, and execution boundary.

A. Workload and expected outcomes

The final workload contains 100 requests per run, comprising 40 valid requests, 30 replay attempts, and 30 tampered-signature attempts. Five repeated runs yield 500 observations per pipeline. For each protected run, the expected counts are 40 valid executions and zero unsafe executions; for each baseline run, the expected counts are 40 valid executions and 60 unsafe executions. These are regression expectations for this specific synthetic dataset. If the request set, randomization strategy, or gate implementation changes, the expected outcomes and the test report should be versioned rather than silently reusing the same headline numbers.

B. Environment and implementation details

A complete independent replication should preserve the exact source revision and the configuration used to generate the reported evidence. At minimum, the record should include runtime and dependency versions, operating system and processor details, storage type, SQLite transaction and journal settings, concurrency configuration, timing method, and the command used to run the benchmark. It should also identify whether database files were recreated between runs, whether the system was warmed up, and whether the timing included audit writes and simulated execution. These details can materially affect both throughput and tail latency. The present manuscript reports the values available in the final evidence output; any missing environmental field should be recorded as missing rather than inferred after the fact.

C. Security interpretation checklist

Before interpreting a new run as evidence of protection, a reviewer should check the following points:

- Valid requests are accepted only when their authenticated representation and policy-relevant fields match the expected request.
- Altered authenticated fields and invalid tags are rejected before the execution stub is called.
- Replaying a consumed identifier does not result in a second simulated execution under the tested persistence conditions.
- Revoked and expired keys are rejected under the lifecycle policy being evaluated.
- Concurrent attempts using the same identifier produce at most one successful reservation in the tested local configuration.
- Failures in reservation or verification are recorded as failures and do not silently fall through to execution.
- Performance reports include valid and unsafe counts as well as latency and throughput, so rejection gains are not confused with indiscriminate blocking.

Passing this checklist would still not demonstrate protection against a compromised signing key, malicious trusted code, a semantically unsafe but correctly authenticated proposal, or a distributed failure between reservation and external execution. Those require separate threat models and experiments. In particular, a test result showing that a request was rejected by a local gate should not be rewritten as a measured reduction in real-world prompt-injection success unless a study directly evaluates prompt-injection attempts and their downstream effects.

References

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. [Online]. Available: https://arxiv.org/abs/2302.12173

[2] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, *Zero Trust Architecture*, NIST Special Publication 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[3] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications: LLM01 Prompt Injection and LLM08 Excessive Agency," 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[4] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Internet Engineering Task Force, Feb. 1997, doi: 10.17487/RFC2104.

[5] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[6] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," arXiv:2307.15043, 2023. [Online]. Available: https://arxiv.org/abs/2307.15043

[7] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," arXiv:2404.13208, 2024. [Online]. Available: https://arxiv.org/abs/2404.13208